

7.API ORM (I)

SISTEMAS DE GESTIÓN EMPRESARIAL
2ºDAM

Bibliografía

[Odoo 11 Development Essentials - Third Edition](#)

ISBN-13: 978-1788477796

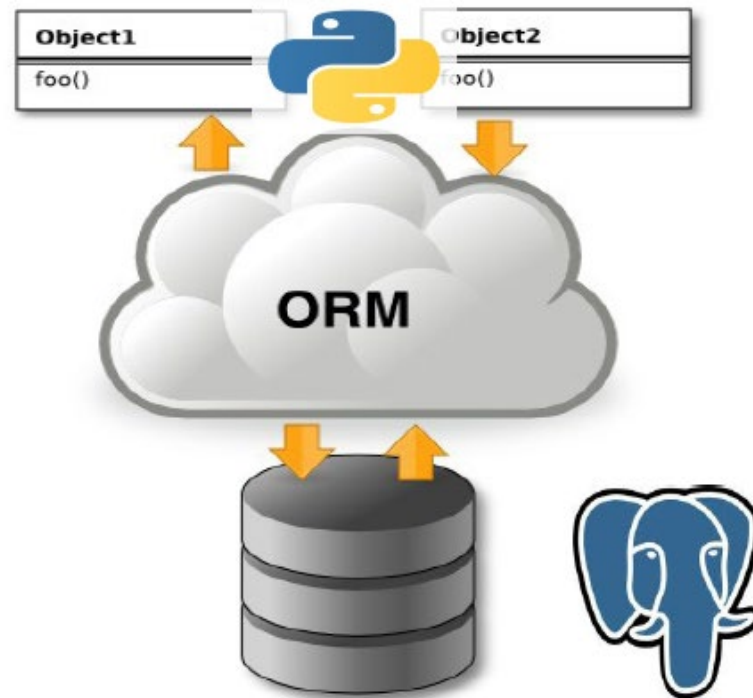
[Odoo 11 Development Cookbook](#)

ISBN-13: 978-1788471817

[Working with Odoo 11 - Third Edition: Configure, manage, and customize your Odoo system](#)

ISBN-13: 978-1788476959

ORM – Object Relational Mapping



Índice

1. Recordsets

2. Decoradores

3. Environment

Recordsets

- Los campos del modelo se pueden leer y escribir directamente desde el registro como atributos.
- El parámetro *self* que tienen todos los métodos, y que de forma normal representa un objeto instanciado del modelo, es realmente un recordset de objetos del modelo.
- Esto es así por la definición que tiene el objeto base *models.Model*

```
class UnModelo(models.Model):  
    _name = 'un.modelo'  
    def un_metodo(self):  
        # self puede contener entre 0 registros o todos  
        # los registros de la base de datos  
        self.do_operation()
```

Recordsets

- Recorrer un conjunto de registros producirá nuevos conjuntos de un solo registro ("singletons"), al igual que recorrer una cadena de Python produce cadenas de un solo carácter

```
def do_operation(self):  
    print (self) # => un.modelo(1, 2, 3, 4, 5)  
    for record in self:  
        print (record) # => un.modelo(1), then un.modelo(2), then un.modelo(3), ...
```

Recordsets: Mapped()

- Los campos del modelo se pueden leer y escribir directamente desde el registro como atributos.
- El método mapped() se usa normalmente cuando se desea acceder a campos no relacionales en un recordset.
- Aplica la función o la expresión proporcionada a cada registro del recordset, devolviendo un recordset si los resultados son recordsets.

```
# devuelve una lista de números resultado de sumar dos campos  
sumList = records.mapped(lambda r: r.field1 + r.field2)
```

```
# devuelve un recordset de socios  
partners = record.mapped('partner_id')
```

Recordsets: Mapped()

```
# devuelve un recordset con los bancos de todos los socios  
# el recordset no tendrá duplicados  
record.mapped('partner_id.bank_ids')
```

```
# devuelve la suma total de cantidades de todos los objetos  
totalCantidad = sum(self.mapped('cantidad'))
```


Recordsets

- También se puede acceder a los valores de campo como elementos de un diccionario.
- Los "singletons" (conjuntos de registros de un solo registro) permiten leer y escribir los campos del modelo directamente desde el registro como atributos.
- Establecer el valor de un campo desencadena una actualización de la base de datos.
- Intentar leer o escribir un campo en un recordset que contenga múltiples registros elevará una excepción.
- Acceder a un campo relacional (Many2One, One2Many, Many2Many) siempre devuelve un conjunto de registro (recordset).

Recordsets

```
>>> record.name
```

Example Name

```
>>> record.company_id.name
```

Company Name

```
>>> record.name = "Bob"
```

```
>>> field = "name"
```

```
>>> record[field]
```

Bob

Recordsets

```
# 3 * len(records) database updates
```

```
for record in records:
```

```
    record.a = 1
```

```
    record.b = 2
```

```
    record.c = 3
```

```
# len(records) database updates
```

```
for record in records:
```

```
    record.write({'a': 1, 'b': 2, 'c': 3})
```

```
# 1 database update
```

```
records.write({'a': 1, 'b': 2, 'c': 3})
```

Recordsets: Caché vs DB

- Odoo mantiene una caché para los campos de los registros.
- La caché facilita que todos los accesos a los campos no emitan una solicitud de base de datos.

`record.name` # Primer acceso obtiene el valor de la DB

`record.name` # Segundo acceso obtiene el valor de la caché

Recordsets: Caché vs DB

Suponga que partners es un recordset de 1000 registros.

¿Cómo se comporta la caché y la DB ante este problema?

```
for partner in partners:  
    print partner.name  
    print partner.lang
```

Recordsets: Caché vs DB

Suponga que partners es un recordset de 1000 registros.

¿Cómo se comporta la caché y la DB ante este problema?

```
countries = set()
for partner in partners:
    country = partner.country_id
    countries.add(country.name)
```

Recordsets

- Los recordsets son inmutables, pero los conjuntos del mismo modelo se pueden combinar usando varios operadores de conjuntos que devuelven nuevos conjuntos de registros.
 - Los operadores de conjuntos no preservan el orden de los registros.
-
- ***record in set*** - devuelve *True* si el registro pertenece al recordset (record debe ser un “singleton”).
 - ***record not in set*** - es la operación inversa a la anterior.
 - ***set1 <= set2*** y ***set1 < set2*** - devuelven *True* si *set1* es un subconjunto de *set2*.
 - ***set1 >= set2*** y ***set1 > set2*** - devuelven *True* si *set1* es un superconjunto de *set2*.
 - ***set1 / set2*** - devuelve un recordset con la unión de los registros de *set1* y *set2*.
 - ***set1 & set2*** - devuelve un recordset con la intersección de los registros de *set1* y *set2*.
 - ***set1 - set2*** – devuelve un recordset con los registros de *set1* que no están en *set2* (operador diferencia)

Recordsets: filtered()

- Devuelve un conjunto de registros que contiene solo registros que satisfacen la función de predicado proporcionada.
- El predicado también puede ser una cadena para filtrar por un campo que sea verdadero o falso.

```
# solo mantiene los registros cuya compañía  
# coincide con la del usuario actual  
records = records.filtered(lambda r: r.company_id == user.company_id)
```

```
# solo mantiene los registros cuyo socio es una compañía  
records = records.filtered("partner_id.is_company")
```


Recordsets: sorted()

- Devuelve un conjunto de registros ordenado por el campo que se especifique.

```
# registros ordenados por el campo nombre  
records = records.sorted(key=lambda r: r.name)
```

Índice

1. Recordsets
- 2. Decoradores**
3. Environment

API ORM – Campos computados

- Los campos se pueden calcular (en lugar de leerlos directamente desde la base de datos) utilizando el parámetro `compute` en la definición del campo.
- Si en el cálculo se utilizan otros campos del modelo se deben especificar mediante el decorador de funciones `@api.depends('campo')`

```
from odoo import api
total = fields.Float(compute='_compute_total')

@api.depends('value', 'tax')
def _compute_total(self):
    for record in self:
        record.total = record.value + record.value * record.tax
```

API ORM – Campos computados II

- Los campos calculados no se almacenan de manera predeterminada, se calculan y devuelven cuando se solicitan.
- Se pueden calcular múltiples campos al mismo tiempo por el mismo método, solo hay que usar el mismo método en todos los campos

```
@api.depends('value', 'discount')
def _apply_discount(self):
    for record in self:
        # compute actual discount from discount percentage
        discount = record.value * record.discount
        record.discount_value = discount
        record.total = record.value - discount
```

Decoradores: depends()

- Es posible que el campo utilizado en el decorado dependa de algún otro campo de la misma tabla o de alguna otra tabla.
- Este decorador se activa cuando el ORM es alterado desde cualquier modelo o formulario. También se activa cuando el formulario ha modificado dicho campo (al pulsar el botón guardar del formulario).
- Cada argumento es una cadena que representa un campo de un modelo.

Decoradores: depends()

- La función se activará si 'partner_id.name' o 'partner_id.is_company' ha sufrido algún cambio en algún lugar formulario del módulo.

```
pname = fields.Char(compute='_compute_pname')
```

```
@api.depends('partner_id.name', 'partner_id.is_company')
def _compute_pname(self):
    for record in self:
        if record.partner_id.is_company:
            record.pname = (record.partner_id.name or "").upper()
        else:
            record.pname = record.partner_id.name
```

Decoradores: onchange()

- Este decorador activa la función decorada cuando un usuario cambia el valor de un campo en un formulario (pero aún no ha guardado el formulario).
- Útil para actualizar automáticamente otros campos en función de ese valor, por ejemplo, actualizar un total final cuando se cambia el impuesto o se agrega una nueva línea de factura.
- **El alcance de este decorador está limitado al mismo modelo/formulario.**
- Si el alcance se encuentra para todo el módulo se debe usar depends().
- Cada argumento es una cadena que representa un campo de un modelo.
- No se admiten campos relacionales del tiempo: gym.name. Únicamente admite nombres de campos simples.
- No es posible utilizar este decorador para un campo del tipo one2many, many2one o many2many.
- Este decorador devuelve un recordset. Sin embargo, únicamente es compatible con el método update() de los métodos CRUD.

Decoradores: onchange()

- La función se activará si 'partner_id' ha sufrido algún cambio en el formulario.

```
@api.onchange('partner_id')
def _onchange_partner(self):
    self.message = "Dear %s" % (self.partner_id.name or "")

return {
    'advertencia' : { 'título' : "Advertencia" , 'mensaje' : "¿Qué es esto?" , 'tipo' : 'notificación'
    },
}
```


Decoradores: depends_context()

- Este decorador es útil cuando se especifica las dependencias del contexto de un método de cálculo no almacenado.
- Ejemplo: Queremos garantizar que un campo calculado se vuelva a calcular según la empresa actual (self.env.company).

```
@api.depends_context('company')
def _compute_display_info(self):
    for record in self:
        record.display_info = record.info + record.company_info
```

Decoradores: constraints()

- Se ejecuta el método cuando el usuario guarda un registro y se utiliza principalmente para realizar una validación de los datos introducidos.
- Se debe especificar cada campo que se vaya a validar como argumento.
- El método se ejecuta cuando al guardar se han modificado el valor de los campos especificados.
- Debería elevar una excepción de tipo **ValidationError** si la validación falla.

```
@api.constrains('name', 'description')
def _check_description(self):
    for record in self:
        if record.name == record.description:
            raise ValidationError("Fields name and description must be different")
```

Decoradores: constraints()

```
from openerp.exceptions import ValidationError

@api.one
@api.constrains('name')
def _check_name_size(self):
    if len(self.name) < 5:
        raise ValidationError('Must have 5 chars!')
```

API ORM – `_sql_constraint`

- El atributo del modelo `_sql_constraints` permite definir una lista de restricciones sobre los registros del modelo, pero se hacen a nivel de la base de datos, utilizando para ello la capacidad de definir restricciones de PostgreSQL.
- Cada restricción está definida por una tupla de tres elementos:
 - Un nombre para identificar la restricción
 - La restricción SQL para usar directamente sobre la base de datos.
 - Un mensaje para informar al usuario cuando se infringe la restricción.
- Para más información sobre restricciones de PostgreSQL en general y restricciones de tabla en particular, eche un vistazo en <http://www.postgresql.org/docs/9.4/static/ddl-constraints.html>.

```
sql_constraints = [  
    ('name_uniq',  
     'UNIQUE (name)',  
     'Book title must be unique.')  
]
```

Decoradores: returns()

- Decorador que permite especificar lo que debe devolverse ya sea un id, ids, False o un RecordSet.

```
@model
@returns('res.partner')
def find_partner(self, arg):
    ...      # devuelve algún registro
```

```
# la salida depende del estilo de la llamada: estilo tradicional vs estilo de registro
partner_id = model.find_partner(cr , uid , arg , context=context)
```

```
# recs = model.browse (cr, uid, ids, context)
partner_record = recs.find_partner(arg)
```

Decoradores: model()

- Decorador que se utiliza para realizar una funcionalidad a nivel de modelo en sí, no para registros.

```
@api.model  
def method(self, args):  
    ...
```

Decoradores: `model_create_multi()`

- Decorador que toma una lista de diccionarios y crea varios registros en función de dicho diccionario.
- El método puede ser llamado con un único diccionario o una lista de diccionarios.

```
record = model.create(vals)
records = model.create([vals, ...])
```

Índice

1. Recordsets
2. Decoradores
- 3. Environment**

Environment

El acceso a la API ORM se hace a través de los modelos (*.py)

- Si queremos acceder al propio modelo

```
self.<función>(....)
```

- Si queremos acceder a otro modelo

```
self.env[<modelo>].<función>(....)
```

Environment

- La variable de entorno *env* almacena diversos datos contextuales utilizados por el ORM: el cursor de la base de datos (para consultas a la base de datos), el usuario actual (para la verificación de derechos de acceso) y el contexto actual (almacenando metadatos arbitrarios).
- El entorno también almacena datos a modo de caché.
- Todos los conjuntos de registros (recordsets) tienen un entorno, que es inmutable, se puede acceder usando *env* y da acceso al usuario actual (*user*), el cursor (*cr*) o al contexto (*context*).

```
>>> records.env
<Environment object ...>
>>> records.env.user
res.user(3)
>>> records.env.cr
<Cursor object ...>
```

Environment

- Al crear un recordset a partir de otro recordset, el entorno se hereda. El entorno se puede usar para obtener un conjunto de registros vacío en otro modelo y consultar ese modelo.

```
recordset_partner = self.env['res.partner']  
resultado = recordset_partner.search([('is_company', '=', True)])  
#resultado res.partner(7, 18, 12, 14, 17)
```

Environment – Low-level SQL

- El atributo *cr*, obtenido como variable de entorno, es el cursor para la transacción actual de la base de datos y permite ejecutar SQL directamente.
- Se utiliza para los casos en que las consultas son difíciles de expresar usando el ORM (por ejemplo consultas con muchos JOIN) o bien por razones de rendimiento.
- Debido a que los modelos usan el mismo cursor y Odoo usa caché de registros, estas memorias caché deben invalidarse al modificar la base de datos mediante el uso de SQL directo.
- Es necesario borrar caché de registros cuando se usa CREATE, UPDATE o DELETE en SQL, pero no SELECT (que simplemente lee la base de datos).
- La eliminación de cachés se puede realizar utilizando el método `invalidate_cache()` del objeto `BaseModel`.

```
self.env.cr.execute("some_sql", param1, param2, param3)  
self.invalidate_cache()
```